Eurobot 2026

Прошивка основной платы робота на STM32F407VGT6

Сводка по проекту: что где лежит, как устроено, где искать при проблемах

Репозиторий: `eurobot2026-stm` · сборка: CMake + Ninja + arm-none-eabi-gcc · ОС: FreeRTOS 10.4.3 · актуально на 23.10.2026

Содержание

1. Что это за проект 2

2. Микроконтроллер и тактирование 3

3. Где что лежит 4

4. Задачи FreeRTOS 5

5. Драйверы и модули 6

6. Протокол с компьютером и команды 7

7. Разъёмы, входы и исполнительные устройства 8

8. Сборка, прошивка, отладка 9

9. Если что-то не так: где смотреть 10

10. Известные проблемы и ловушки 11

1. Что это за проект

Прошивка «нижнего уровня» робота Eurobot 2026. Плата управления на STM32F407 получает команды от бортового компьютера (верхний уровень, HL) по UART и выполняет их: двигает колёсную базу, считает одометрию, управляет захватами (сервоприводы Dynamixel, сервы через PCA9685, шаговый подъёмник), насосами и клапанами, читает кнопки, концевики и батарею.

Общая схема работы:

```
Компьютер робота (reset10: контейнер communication или пульт test_low)
    │  USART1, 921600 бод, протокол HL-RTU (FF FF len cmd data crc)
    ▼
STM32F407: TerminalTask разбирает команды
    ├─► OdometryTask ─► 3 мотора Maxon (USART6, RS-485)
    ├─► GripperTask  ─► Dynamixel AX (USART3), серво PCA9685 (I2C2),
    │                   насосы и клапаны (SPI GPO)
    ├─► StepperTask  ─► шаговый подъёмник (L6474) + концевик
    └─► ответы HL: скорость, координаты, батарея (UART4), кнопки
```

- **Робот**: голономная база на трёх омни-колёсах с моторами Maxon, два захвата для блоков, подъёмник на шаговом двигателе, рейка с Dynamixel, насосы и клапаны.
- **Кто главный**: стратегию и навигацию ведёт компьютер. Прошивка исполняет микрокоманды и отдаёт телеметрию.
- **Матч**: `GameTask` ждёт выдёргивания стартового шнура, через 99 с останавливает всё по правилам Eurobot.

2. Микроконтроллер и тактирование

Параметр	Значение
МК	STM32F407VGT6, Cortex-M4F с FPU (hard float, fpv4-sp-d16)
Память	Flash 1 МБ (0x08000000), SRAM 128 КБ (0x20000000), CCM 64 КБ (не используется)
Кварц	HSE 16 МГц (`HSE_VALUE` в `Core/App/system_stm32f4xx.c`)
SYSCLK	168 МГц: 16 / 8 × 168 / 2 (`board_hw_config.c`, `SystemClockConfig`)
Шины	AHB 168 МГц; APB1 = 42 МГц (÷4), таймеры APB1 84 МГц; APB2 = 84 МГц (÷2), таймеры APB2 168 МГц
Сторожевой таймер	IWDG есть (`IWDG_Init`), но отключён в `main.c`
Отладка	SWD через ST-LINK V2 (разъём SWD на плате: GND, NRST, SWCLK, SWDIO). NRST к ST-LINK не подключён
Размер прошивки	Debug ≈ 69 КБ, Release ≈ 43 КБ Flash; RAM ≈ 21,7 КБ

Частоты APB1 = 42 МГц и APB2 = 84 МГц важны для расчёта делителей. `LL_USART_SetBaudRate` и `LL_I2C_ConfigSpeed` получают частоту шины явным числом. При смене делителей эти числа нужно поправить вручную.

Используемая периферия

Интерфейс	Пины	Скорость	Что подключено	Где настраивается
USART1	PA9 TX, PA10 RX	921600, 8N1, DMA	Компьютер (HL), протокол HL-RTU. Разъём USART1	`USART1_TerminalInit`
USART3	PD8 полудуплекс, PD9 DE	1 Мбит/с	Шина Dynamixel AX (разъёмы GND/+12V/DIO справа)	`USART3_DynamixelInit`
USART6	RS-485	2 Мбит/с	Контроллеры моторов Maxon (ID 0x21–0x23)	`USART6_RS485_Init`
UART4	RS-485, DE PA15	2 Мбит/с	Плата BMS (ID 0x11)	`UART4_RS485_Init`
UART5	TX/RX	115200	Отладочный терминал (Serial)	`UART5_SerialInit`
USART2	RS-485, DE PD6	—	Пины настроены, в прошивке не используется	`board_gpio_config.c`
I2C2	PB10 SCL, PB11 SDA	100 кГц	PCA9685 (адрес 0x40), 16 каналов серво	`I2C2_Init`
SPI1	SCK/MOSI, защёлка PA4	—	74HC595 (силовые выходы PO1–PO20) и драйверы L6474	`SPI1_Board_HW_Init`
TIM1	—	—	Импульсы STEP шаговых двигателей	`TIM1_Steppers_PWM_Init`
TIM4 / TIM5	PWM1–4 / PWM5–8	—	ШИМ-выходы (RC-серво, H-мост)	`TIM4_PWM_Group1234_Init`
TIM8	—	—	Светодиодная лента WS2812B	`TIM8_WS2812B_Init`
TIM14	—	—	Тайм-ауты приёма UART-протоколов	`TIM14_USARTx_TimeoutInit`

3. Где что лежит

Путь	Что внутри
`Core/App/`	Логика верхнего уровня прошивки: `main.c` (старт), `terminal*.c` (команды от HL), `kinematics.c`, `odometry.c`, `robot_behavior_routine.c` (кнопки, Maxon вкл/выкл, аварийный стоп), `bus_device.c` (ID устройств на шинах), `stm32f4xx_it.c` (прерывания)
`Core/App/Include/`	`terminal_cmd_table.h` — коды команд; `robot_gpio_defines.h` — назначение входов (кнопки, концевики)
`Core/BSP/`	Работа с железом платы: такты и периферия (`board_hw_config.c`), пины (`board_gpio_config.c`), I2C2, SPI GPO, L6474, шаговики, ШИМ, WS2812B, драйвер UART с DMA
`Core/BSP/Include/`	`board_gpio_defines.h` — все пины платы с именами разъёмов; `spi_gpo_defines.h` — биты силовых выходов
`Core/Lib/`	Драйверы устройств и протоколов: Dynamixel, Maxon, PCA9685, BMS, кнопки, HL-RTU, FIFO, H-мост, RC-серво
`Core/OS_Tasks/`	Задачи FreeRTOS (одна задача — один файл). `template_task.c` — шаблон, не собирается
`Drivers/`	CMSIS, STM32F4 LL-драйверы, CMSIS-DSP (матрицы, sin/cos). Оставлены только используемые файлы
`Middlewares/FreeRTOS-Kernel/`	Ядро FreeRTOS, порт `GCC/ARM_CM4F`, `include/FreeRTOSConfig.h`
`cmake/`	`sources.cmake` (список исходников), toolchain, linker script `STM32F407VG_FLASH.ld`, `openocd.cmake` (прошивка), `check_firmware.py` (проверка образа)
`tools/`	`monitor_stm32.py` (проверка платы по SWD), `serial_read_test.py` (проверка UART)
`.vscode/`	Задачи сборки и прошивки, отладка Cortex-Debug + OpenOCD, кнопки в статус-баре
`Makefile`	Короткие команды: make, make release, make flash, make flash-remote …
`README.md`, `docs/USAGE_RU.md`	Инструкция по сборке и прошивке, история исправлений, чек-лист проверок
`CHANGELOG.md`	Что и почему менялось, найденные проблемы и расследования
`MIGRATION_REPORT.md`	Подробный отчёт о переносе с Keil на CMake/GCC (история)

Порядок запуска (`Core/App/main.c`)

```
SYSCFG_PWR_Init → SystemClockConfig (168 МГц) → HW_ClockConfig
Board_GPIO_Config → Robot_GPIO_Config          пины платы и входы робота
I2C2_Init → PCA9685.Init(0x40)                 результат в PCA9685_InitStatus (0 = OK)
SPI1_Board_HW_Init → SPI_GPO_ResetOutputPin    все силовые выходы выключены
USART1/3/6, UART4/5 init                       + привязка протоколов (BindProtocol)
TIM1, TIM4, TIM5, TIM8, TIM14 init
CreateTask(...)  →  vTaskStartScheduler()
```

4. Задачи FreeRTOS

FreeRTOS 10.4.3, вытесняющий планировщик, тик 1 мс, 7 уровней приоритета, **только статическое выделение памяти** (`configSUPPORT_DYNAMIC_ALLOCATION = 0`). Стек указан в словах (×4 байта).

Задача (файл)	Приор.	Стек	Что делает
ButtonsPollTask `buttons_poll_task.c`	3	64	Опрашивает 13 входов (4 кнопки + 9 концевиков) с антидребезгом 8 мс, кладёт события в очередь
ButtonsCallbackTask `buttons_callback_task.c`	2	64	Достаёт события из очереди, вызывает `ButtonsCallback` → выставляет флаги в `robot_ctrl`
OdometryTask `odometry_task.c`	1	512	Периодически пересчитывает одометрию и скорость колёс (`Odometry.UpdatePath`)
TerminalTask `terminal_task.c`	1	512	Берёт кадры из очереди USART1 и вызывает обработчик из `terminal_cmd_handlers[]`
GameTask `game_task.c`	1	512	Логика матча: ждёт стартовый шнур, через 99 с — `MaxonsStop`, `StopAllActuators`, `COMM_ENABLE = 0`
StepperTask `stepper_task.c`	1	512	Шаговый подъёмник: калибровка по концевику `LINEAR_SWITCH_2`, включение/выключение драйвера
GripperTask `gripper_task.c`	1	512	Исполняет команды захватов (`gripper_cmd`): Dynamixel, PCA9685, насосы, клапаны, движения. Будит TerminalTask по завершении
DemoTask `demo_task.c`	—	512	**Не создаётся.** Демо-режимы для проверки железа (выходы, серво, шаговики)
DebugTerminalTask `debug_task.c`	—	512	**Не создаётся.** Отладочный терминал на UART5
Idle	0	—	Служебная задача FreeRTOS

Правило: в цикле любой задачи с приоритетом выше 1 обязательно должен быть `vTaskDelay` или ожидание очереди. Пустой `while(1)` у DemoTask с приоритетом 4 полностью блокировал TerminalTask: плата не отвечала даже на PING. Цикл с `taskYIELD()` отдаёт процессор только задачам того же приоритета.

Как задачи общаются

- `robot_ctrl` (`robot_behavior_routine.h`) — общее состояние: стартовый шнур, сторона поля, стратегия, Maxon вкл/выкл, концевики.
- `gripper_cmd` и параметры (`gripper_task.c`) — терминал пишет номер команды, GripperTask исполняет. Терминал ждёт завершения через `WaitForGripperCompletion` (уведомление задачи, тайм-аут 5 с).
- `stepper1_calibrate`, `stepper1_goal_position` — управление подъёмником из терминала и захвата.
- Очередь событий кнопок `button_callbacks_queue_handle` — между ButtonsPollTask и ButtonsCallbackTask.

5. Драйверы и модули

Файл	Назначение	Ключевые функции / заметки
BSP/board_hw_config.c	Такты, UART, I2C2, SPI1, таймеры, IWDG	Все *_Init, скорости UART
BSP/board_gpio_config.c	Режимы всех пинов платы	Альтернативные функции, подтяжки
BSP/usartx_communication.c	Общий драйвер UART: DMA, IDLE, полудуплекс (DE)	BindProtocol, USARTx_IRQ_Process
BSP/usartx_timeout.c	Тайм-ауты приёма на TIM14	Нужны протоколу Dynamixel
BSP/i2c2.c	Блокирующий I2C2 для PCA9685	Каждое ожидание ≤ 20 мс, коды ошибок −2…−6
BSP/spi_gpo.c	Силовые выходы через 74HC595	SPI_GPO_SetOutputPin/ResetOutputPin, теневой регистр SPI_GPO_REG
BSP/l6474_drvr.c, stepper.c	Драйвер L6474 (SPI) и генерация шагов (TIM1)	StepToPosisition, SetZeroPosisition; шаги идут в прерывании
BSP/pwm.c, ws2812b.c	ШИМ-выходы и адресная лента	TIM4/5, TIM8 + DMA
Lib/hl_rtu.c	Протокол с компьютером (USART1)	Кадр FF FF len cmd data crc; очередь запросов
Lib/dynamixel_master.c, dynamixel_ax.c, _mx.c	Протокол Dynamixel 1.0 (USART3, а также USART6/UART4)	Тайм-аут ответа 2 мс + повторы; ID 0xFE = широковещательный
Lib/maxon.c	Контроллеры колёс Maxon	Set_TPM, скорости, тики энкодеров
Lib/pca9685.c	16-канальный ШИМ для серво (I2C)	SetServoAngleWithAddr; 50 Гц, 500–2500 мкс
Lib/bms_board.c	Плата батареи (UART4)	Напряжение, калибровка
Lib/button.c	Кнопки с антидребезгом	Состояния PRESSED/RELEASED, CLICK/HOLD
Lib/rc_servo.c, h_bridge.c	RC-серво и H-мост на ШИМ	Используются в демо-режимах
Lib/serial.c	Простой байтовый UART (UART5)	Отладочный терминал
Lib/fifo_routine.c	Кольцевые буферы	Очереди кадров UART
App/kinematics.c	Кинематика трёх омни-колёс	Радиус колеса 29,5 мм, коэффициенты колёс
App/odometry.c	Одометрия и задание скорости базы	SetSpeed, GetCoord

6. Протокол с компьютером и команды

USART1, 921600 бод, 8N1. Запрос и ответ имеют одинаковый формат: `FF FF | len | cmd | data… | crc`, где `len = длина data + 2`, `crc = ~(len + cmd + Σdata) & 0xFF`. Например, PING: `FF FF 02 00 FD`. Плата отвечает на каждую команду кадром с тем же `cmd`. Коды — `Core/App/Include/terminal_cmd_table.h`, обработчики — `Core/App/terminal_cmd_handlers.c`.

Код	Команда	Что делает
0x00	Ping	Проверка связи
0x04 / 0x05	Set/GetSpeed	Скорость базы dx, dy, dphi (float)
0x06 / 0x07	Set/GetCoord	Координаты робота
0x08 / 0x09	GetBatteryState / BMS calib	Батарея (UART4)
0x0A / 0x0B	Get/Set AX angle	Угол Dynamixel
0x0D / 0x0E	Reset/Get wheel ticks	Тики колёс
0x0F, 0x21, 0x22	Pump / Pumps / Valves	Насосы и клапаны
0x12	GetToggle	Сторона поля (1 = левая)
0x13	GetDebugMessage	Текстовое сообщение 32 байта
0x14	GetLinearSwitchesState	9 концевиков
0x15	SetMaxonsState	Включить/выключить моторы колёс
0x16	SetCommState	Включить/выключить исполнение команд (COMM_ENABLE)
0x17–0x19	Reset / GetReset / ClearReset	Флаг сброса платы
0x1C	GrabRotate	Поворот блоков захватом
0x1D	PCA9685 SetServoAngle	Отладка серво (в отладочном терминале)
0x1E	ToggleCursor	Dynamixel-курсор
0x1F / 0x20	SideGrips / GripperLift	Боковые захваты / подъём Dynamixel
0x23	GetWheelPath	Пройденный путь колеса
0x25	Move	Движение по времени и скорости
0x26	GetKey	Стартовый шнур
0x27	GripperVertical	Высота подъёмника (мм, шаговик)
0x28	RotateWheelOnce	Калибровка колеса
0x29 / 0x2D / 0x2E	MoveRelative / …Poll / GetMoveStatus	Относительное перемещение
0x2A	FreeWheels	Отпустить колёса
0x2B / 0x2C	BlocksShift / BlocksRotateMask	Сдвиг и поворот блоков
0x2F	GripperSlide	Рейка (Dynamixel ID 13)

После `COMM_ENABLE = 0` (конец матча или команда 0x16) плата **отвечает** на команды, но не исполняет движения. Если плата не отвечает совсем, проблема не в COMM_ENABLE (см. раздел 9).

7. Разъёмы, входы и исполнительные устройства

Как читать распиновку платы

- **SPI2, SPI3, I2C1, I2C2**: число на шелкографии = «Connector Pin» в таблице этого разъёма. Пин 1 — GND, пин 8 — +5V.
- **28 GPIO**: ряд = `+5V | нечётный | чётный | +5V | GND`. Число = «Board Pin Number» → GPIOxx → ножка STM32.
- **Shared with**: одна ножка МК выведена на два разъёма — электрически это один провод.
- Имя в коде ищется в `board_gpio_defines.h` (например, `GPIO26_PC13`), назначение — в `robot_gpio_defines.h`.
- Входы GPIO на плате: сигнал → резистор 1 кОм → ножка МК, конденсатор 100 нФ на GND, диод на +5V.

Входы робота

Вход	STM32	Разъём	Смысл
`STARTING_CORD_BTN`	PC15	I2C2 п.5 = 28 GPIO п.27	Стартовый шнур (1 = вставлен)
`PLAYGROUND_SIDE_BTN`	PE6	I2C2 п.6 = 28 GPIO п.25	Сторона поля
`STRATEGY_BTN`	PE5	I2C2 п.4 = 28 GPIO п.23	Выбор стратегии (клик)
`MAXON_ENABLE_BTN`	PE2	I2C2 п.7 = 28 GPIO п.21	Разрешение моторов колёс
`LINEAR_SWITCH_1`	PC14	28 GPIO п.28	Концевик 1 (в логике движения не используется)
`LINEAR_SWITCH_2`	PC13	28 GPIO п.26	**Концевик подъёмника** (калибровка шаговика)
`LINEAR_SWITCH_3…9`	PE4, PE3, PE0, PB7, PB4, PB3, PD3	28 GPIO п.24, 22, 20, 18, 16, 14, 12	Концевики 3–9 (передаются на HL)

Все входы подтянуты к GND, активный уровень — высокий. Концевики нормально замкнутые: замкнут → 1 → «не нажат».

Исполнительные устройства

Устройство	Подключение	Адресация
Моторы колёс Maxon ×3	USART6, RS-485	ID 0x22, 0x21, 0x23 (`bus_device.c`)
Dynamixel AX	USART3, разъёмы GND/+12V/DIO (одна шина)	Захват 1: подъём 254*, 112; боковые 7, 8. Захват 2: подъём 1, 2; боковые 16, 25. Рейка 13, курсор 98
Серво через PCA9685	I2C2, адрес 0x40	Захват 1: CH1–4 блоки, CH5/6 сдвиг; захват 2: CH7–10, CH11/12
Шаговый подъёмник	Stepper Output 1 (L6474)	25 шагов/мм, ноль по LINEAR_SWITCH_2
Насосы, клапаны	Power Output 1–20 через 74HC595	Биты в `spi_gpo_defines.h`
Батарея	UART4, RS-485	BMS, ID 0x11

* ID 254 (0xFE) — широковещательный адрес Dynamixel: команда уходит всем сервам, а ответа нет. Нужно заменить на реальный ID (см. CHANGELOG).

8. Сборка, прошивка, отладка

Действие	Команда
Собрать Debug / Release	`make` / `make release`
Проверить образ	`make check` / `make check-release`
Прошить через ST-LINK на этом компьютере	`make flash` / `make flash-release` (OpenOCD)
Собрать Release и прошить через reset10	`make flash-remote` (scp + `~/flash_stm32.sh`)
Сброс МК	`make reset`
Проверка живой платы по SWD	`make monitor`
Очистить	`make clean`
Результаты	`build/debug/firmware.{elf,bin,hex,map}`, аналогично `build/release/`

- Новый `.c` файл добавляется в `EUROBOT_SOURCES` в `cmake/sources.cmake`.
- VS Code: Cmd+Shift+B — сборка; F5 — «Debug (OpenOCD)» (прошивка, остановка на main, список задач FreeRTOS); «Attach (OpenOCD)» — без прошивки.
- Требуется: CMake ≥ 3.24, Ninja, ARM GNU Toolchain, OpenOCD; для reset10 — ssh-доступ `ssh reset10`.

На компьютере робота (reset10)

```
ssh reset10                                    # вход по ключу
st-info --probe                                # видит ли ST-LINK микроконтроллер (chipid 0x0413)
cd ~/test_low && bash start.sh                 # пульт управления (останавливает контейнер communicati
cd ~/test_low && .venv/bin/python check_uart.py  # только PING
cd ~ && ./flash_stm32.sh                       # прошить ~/firmware.hex через st-flash
```

9. Если что-то не так: где смотреть

Симптом	Сначала проверить	Где смотреть
Плата не отвечает даже на PING	1) `st-info --probe`: виден ли чип. 2) `dmesg` на reset10: не переподключался ли CP2102. 3) Не держит ли порт контейнер communication. 4) По SWD: какая задача выполняется (`pxCurrentTCB`)	`terminal_task.c`, приоритеты в `main.c`, `demo_task.c`
Отвечает, но не едет	`COMM_ENABLE` (прошло 99 с после выдёргивания шнура?), `maxons_state`, питание Maxon	`game_task.c`, `robot_behavior_routine.c`, `terminal_cmd_handlers.c`
ST-LINK не видит чип (chipid 0x0000)	Питание 3.3V, GND, провода SWD. После горячего отключения нагрузки — передёрнуть питание	Железо; CHANGELOG, раздел о расследовании
Серво PCA9685 не двигаются	`PCA9685_InitStatus`: 0 OK, −2 тайм-аут, −3 нет ответа, −4 ошибка шины	`i2c2.c`, `pca9685.c`, `main.c`
Dynamixel не отвечает	ID, питание 12 В, одна шина для всех DIO, ID 254 = широковещательный	`gripper_task.c` (таблица захватов), `dynamixel_master.c`
Подъёмник едет вниз без остановки	Концевик 28 GPIO п.26 (PC13); калибровка запускается при каждом старте	`stepper_task.c`, строка `stepper1_calibrate = 1` в `gripper_task.c`
Насос/клапан не включается	Бит в `SPI_GPO_REG`, питание Power Output	`spi_gpo.c`, `spi_gpo_defines.h`
Неверные координаты	Коэффициенты колёс, радиус, тики	`kinematics.c`, `odometry.c`, `maxon.c`
HardFault / перезагрузки	По SWD: CFSR `0xE000ED28`, HFSR `0xE000ED2C`, причина сброса RCC_CSR `0x40023874`	`stm32f4xx_it.c`, стеки задач

Чтение памяти платы по SWD без остановки

Адреса переменных зависят от сборки — берите их из ELF той прошивки, что стоит на плате:

```
arm-none-eabi-nm build/release/firmware.elf | grep -E " (xTickCount|pxCurrentTCB)$"
# пример: прочитать 4 байта по адресу из nm
ssh reset10 'st-flash --hot-plug read /tmp/x.bin 0x2000497c 4'
```

- `xTickCount` растёт ~1000/с — FreeRTOS работает; сбросился — была перезагрузка.
- `pxCurrentTCB` → имя задачи по смещению 0x34 в TCB — кто сейчас занимает процессор.
- Сверить прошивку: прочитать Flash с `0x08000000` длиной `firmware.bin` и сравнить SHA-256.

Безопасность при работе с железом

- Разъёмы с питанием нагрузки (+5V, +12V, серво) подключать и отключать только при выключенном питании.
- Перед тестами движения поднять робота так, чтобы колёса не касались пола, и держать под рукой аварийное отключение силового питания.
- Калибровку подъёмника можно временно отключить, закомментировав `stepper1_calibrate = 1` в `gripper_task.c`. Тогда нулём считается положение на момент включения.

10. Известные проблемы и ловушки

Что	Последствие	Где
Задача с пустым циклом и высоким приоритетом	Блокирует все задачи ниже, в том числе терминал	`demo_task.c` (DemoTask не создаётся)
ID Dynamixel 254 у захвата 1	Команды уходят всем сервам, ожидание ответа всегда до тайм-аута	`gripper_task.c`, таблица захватов
`SPI_GPO_REG |= …` вне `vTaskSuspendAll`	Гонка между задачами: изменение выхода может потеряться	`spi_gpo.c`
Калибровка ищет концевик до −100000 шагов	Если концевик не сработает, подъёмник уедет до упора	`stepper_task.c`
Стартовый шнур не подключён (вход = 0)	Матч стартует сразу, через 99 с команды движения игнорируются	`game_task.c`
`strategy++` в условном выражении	Неопределённое поведение (предупреждение компилятора)	`robot_behavior_routine.c`
`move_speed` может быть не инициализирован	Непредсказуемая скорость при неверном направлении	`gripper_task.c`
Горячее отключение нагрузки	МК пропадает по SWD до передёргивания питания (вероятно, питание/земля)	CHANGELOG, расследование
Стек ButtonsPoll/Callback — 64 слова	Мало: при усложнении обработчика возможно переполнение	`buttons_*_task.c`

Документы

- `README.md` — быстрый старт и структура.
- `docs/USAGE_RU.md` — сборка, прошивка, история исправлений I2C, чек-лист стендовых проверок.
- `CHANGELOG.md` — изменения, найденные ошибки, расследования, таблица подключения серв.
- `MIGRATION_REPORT.md` — подробности переноса с Keil.
- `CodeStyle.txt` — стиль: функции PascalCase, переменные snake_case, `TIM14_PWM_Init()`.